

EXPERIMENT 1B

PERFORMING CRISP-DM AND SEMMA

Aim :

To apply the CRISP-DM (Cross-Industry Standard Process for Data Mining) and SEMMA (Sample, Explore, Modify, Model, Assess) methodologies for systematically understanding, preparing, analyzing, modeling, and evaluating data to extract meaningful insights and support effective decision-making.

Algorithm :

1. Apply the **CRISP-DM** framework by understanding the problem, collecting and preparing the data, building the model, evaluating the results, and presenting the findings.
2. Apply the **SEMMA** framework by sampling the data, exploring its characteristics, modifying and preprocessing it, building the model, and assessing its performance.
3. Compare and interpret the outcomes obtained using both **CRISP-DM** and **SEMMA** methodologies.
4. Draw conclusions based on the analysis and communicate the results for informed decision-making.

Code - CRISP-DM:

1. TO LOAD THE DATASET

```
In [1]: import pandas as pd  
df = pd.read_csv("Instagram_Analytics.csv")
```

```
In [2]: df.head()
```

```
Out[2]:
```

	post_id	account_id	account_type	follower_count	media_type	content_category	traffic_source
0	IG0000001	7	brand	3551	reel	Technology	Home Feed
1	IG0000002	20	creator	31095	image	Fitness	Hashtag
2	IG0000003	15	brand	8167	reel	Beauty	Reels Feed
3	IG0000004	11	creator	9044	carousel	Music	External
4	IG0000005	8	creator	15986	reel	Technology	Profile

5 rows × 23 columns

2. TO PRINT TAIL AND ANALYSE SHAPE

```
In [3]: df.tail()
```

```
Out[3]:
```

	post_id	account_id	account_type	follower_count	media_type	content_category	traffic_s
29994	IG0029995	5	brand	10739	carousel	Travel	Reel
29995	IG0029996	3	brand	10018	image	Beauty	He
29996	IG0029997	18	creator	7486	image	Photography	f
29997	IG0029998	6	creator	10034	carousel	Technology	f
29998	IG0029999	9	creator	6916	image	Technology	f

5 rows × 23 columns

```
In [4]:
```

```
print("Rows:", df.shape[0])  
print("Columns:", df.shape[1])
```

```
Rows: 29999  
Columns: 23
```

3. TO PRINT INFO OF THE DATASET

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 29999 entries, 0 to 29998  
Data columns (total 23 columns):  
#   Column                Non-Null Count  Dtype  
---  -  
0   post_id                29999 non-null  object  
1   account_id             29999 non-null  int64  
2   account_type           29999 non-null  object  
3   follower_count          29999 non-null  int64  
4   media_type              29999 non-null  object  
5   content_category        29999 non-null  object  
6   traffic_source           29999 non-null  object  
7   has_call_to_action      29999 non-null  int64  
8   post_datetime           29999 non-null  object  
9   post_date               29999 non-null  object  
10  post_hour                29999 non-null  int64  
11  day_of_week              29999 non-null  object  
12  likes                    29999 non-null  int64  
13  comments                 29999 non-null  int64  
14  shares                   29999 non-null  int64  
15  saves                    29999 non-null  int64  
16  reach                    29999 non-null  int64  
17  impressions              29999 non-null  int64  
18  engagement_rate          29999 non-null  float64  
19  followers_gained          29999 non-null  int64  
20  caption_length            29999 non-null  int64  
21  hashtags_count            29999 non-null  int64  
22  performance_bucket_label  29999 non-null  object  
dtypes: float64(1), int64(13), object(9)  
memory usage: 5.3+ MB
```

4. TO PRINT COLUMN TO LIST

```
: print("Columns in Dataset:\n")
print(df.columns.tolist())
```

Columns in Dataset:

```
['post_id', 'account_id', 'account_type', 'follower_count', 'media_type', 'content_category', 'traffic_source', 'has_call_to_action', 'post_datetime', 'post_date', 'post_hour', 'day_of_week', 'likes', 'comments', 'shares', 'saves', 'reach', 'impressions', 'engagement_rate', 'followers_gained', 'caption_length', 'hashtags_count', 'performance_bucket_label']
```

5. PRINT SAMPLE

```
In [7]: df.sample(5, random_state=42)
```

Out[7]:

	post_id	account_id	account_type	follower_count	r
2308	IG0002309	11	brand	9044	
22404	IG0022405	17	creator	15923	
23397	IG0023398	13	brand	5824	
25058	IG0025059	17	creator	15923	
2664	IG0002665	2	creator	22291	

6. TO DESCRIBE THE DATASET

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%
account_id	29999.0	10.567819	5.763952	1.0	6.000	11.0000	16.000
follower_count	29999.0	10278.305477	6691.862669	3083.0	5824.000	9044.0000	10739.000
has_call_to_action	29999.0	0.348778	0.476592	0.0	0.000	0.0000	1.000
post_hour	29999.0	11.499417	6.900587	0.0	6.000	12.0000	17.000
likes	29999.0	287.653588	317.647682	0.0	104.000	199.0000	363.000
comments	29999.0	8.521917	10.116505	0.0	3.000	6.0000	11.000
shares	29999.0	14.426614	16.420899	0.0	5.000	10.0000	19.000
saves	29999.0	42.517284	47.808844	0.0	15.000	29.0000	54.000
reach	29999.0	6272.475449	4985.877059	268.0	3058.000	4913.0000	7863.000
impressions	29999.0	8466.952498	6825.900355	327.0	4102.000	6595.0000	10620.500
engagement_rate	29999.0	0.042107	0.024095	0.0	0.024	0.0406	0.057
followers_gained	29999.0	502.152805	290.364038	0.0	251.000	501.0000	755.000
caption_length	29999.0	120.081169	11.009108	70.0	113.000	120.0000	127.000
hashtags_count	29999.0	7.987933	2.828130	0.0	6.000	8.0000	10.000

7. TO ANALYSE THE MISSING VALUES

```
missing = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Percentage": (df.isnull().sum()/len(df))*100
})
missing.sort_values("Missing Values", ascending=False)
```

	Missing Values	Percentage
post_id	0	0.0
likes	0	0.0
hashtags_count	0	0.0
caption_length	0	0.0
followers_gained	0	0.0
engagement_rate	0	0.0
impressions	0	0.0
reach	0	0.0
saves	0	0.0
shares	0	0.0
comments	0	0.0
day_of_week	0	0.0
account_id	0	0.0
post_hour	0	0.0
post_date	0	0.0
post_datetime	0	0.0

8. TO PRINT DUPLICATE ROWS

```
duplicates = df.duplicated().sum()
print(f"Duplicate Rows: {duplicates}")
```

Duplicate Rows: 0

9. TO PRINT UNIQUE VALUES

```
for column in df.columns:  
    print(f"{column}: {df[column].nunique()} unique values")
```

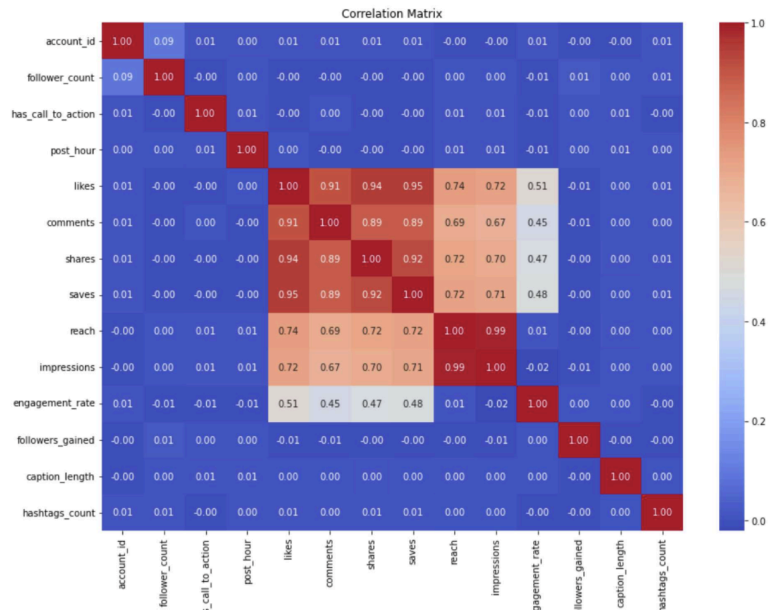
```
post_id: 29999 unique values  
account_id: 20 unique values  
account_type: 2 unique values  
follower_count: 20 unique values  
media_type: 3 unique values  
content_category: 10 unique values  
traffic_source: 6 unique values  
has_call_to_action: 2 unique values  
post_datetime: 8475 unique values  
post_date: 366 unique values  
post_hour: 24 unique values  
day_of_week: 7 unique values  
likes: 1585 unique values  
comments: 110 unique values  
shares: 157 unique values  
saves: 377 unique values  
reach: 11965 unique values  
impressions: 14315 unique values  
engagement_rate: 1243 unique values  
followers_gained: 1001 unique values  
caption_length: 87 unique values  
hashtags_count: 22 unique values  
performance_bucket_label: 4 unique values
```

10. TO VISUALISE CORRELATION MATRIX

```

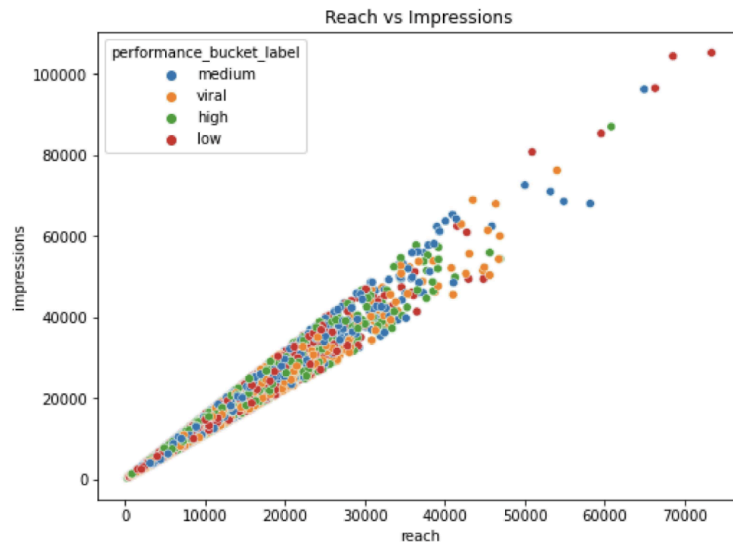
import matplotlib.pyplot as plt
import seaborn as sns
numeric_df = df.select_dtypes(include=['number'])
plt.figure(figsize=(14,10))
sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Matrix")
plt.show()

```



11. TO ANALYSE REACH VS IMPRESSION

```
plt.figure(figsize=(8,6))
sns.scatterplot(
    data=df,
    x='reach',
    y='impressions',
    hue='performance_bucket_label'
)
plt.title("Reach vs Impressions")
plt.show()
```



12. DATA CLEANING AND PREPROCESSING

```
: df_clean = df.copy()
```

```
:
num_cols = df_clean.select_dtypes(include=['number']).columns
df_clean[num_cols] = df_clean[num_cols].fillna(df_clean[num_cols].median())
cat_cols = df_clean.select_dtypes(include=['object']).columns
df_clean[cat_cols] = df_clean[cat_cols].fillna(df_clean[cat_cols].mode().iloc[0])
```

```
: print("Duplicates before:", df_clean.duplicated().sum())
df_clean.drop_duplicates(inplace=True)
print("Duplicates after:", df_clean.duplicated().sum())
```

```
Duplicates before: 0
Duplicates after: 0
```

```
: df_clean['post_datetime'].dtype
```

```
: dtype('O')
```

13. EXTRACT DATA TIME AND CONVERTING THEIR FORMATS

```
df_clean['post_datetime'] = pd.to_datetime(
    df_clean['post_datetime'],
    errors='coerce'
)
df_clean['post_date'] = pd.to_datetime(
    df_clean['post_date'],
    errors='coerce'
)
```

```
print(df_clean[['post_datetime', 'post_date']].dtypes)
```

```
post_datetime    datetime64[ns]
post_date        datetime64[ns]
dtype: object
```

```
df_clean['year'] = df_clean['post_datetime'].dt.year
df_clean['month'] = df_clean['post_datetime'].dt.month
df_clean['day'] = df_clean['post_datetime'].dt.day
df_clean['hour'] = df_clean['post_datetime'].dt.hour
df_clean['day_of_week'] = df_clean['post_datetime'].dt.day_name()
```

```
df_clean['post_datetime'].dtype
```

```
dtype('<M8[ns]')
```

```
df_clean['post_datetime'].head(10)
```

```
0    2024-11-30 06:00:00
1    2025-08-15 15:00:00
2    2025-09-11 16:00:00
3    2025-09-18 03:00:00
4    2025-03-21 09:00:00
5    2025-05-06 08:00:00
6    2025-09-21 09:00:00
7    2025-01-06 18:00:00
8    2025-10-13 05:00:00
9    2025-03-10 11:00:00
Name: post_datetime, dtype: datetime64[ns]
```

14. TO PRINT DATA TIME AFTER CONVERSION

```
df_clean['post_datetime'].head(10)
```

```
0    2024-11-30 06:00:00
1    2025-08-15 15:00:00
2    2025-09-11 16:00:00
3    2025-09-18 03:00:00
4    2025-03-21 09:00:00
5    2025-05-06 08:00:00
6    2025-09-21 09:00:00
7    2025-01-06 18:00:00
8    2025-10-13 05:00:00
9    2025-03-10 11:00:00
Name: post_datetime, dtype: datetime64[ns]
```

```
df_clean.drop(columns=['post_id'], inplace=True)
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
categorical_cols = df_clean.select_dtypes(include='object').columns
for col in categorical_cols:
    df_clean[col] = le.fit_transform(df_clean[col])
```

15. FEATURE AND TARGET VARIABLE SEPARATION


```
X = df_clean.drop('performance_bucket_label', axis=1)
y = df_clean['performance_bucket_label']
```

```
print(X.dtypes)
```

```
account_id          int64
account_type        int32
follower_count      int64
media_type          int32
content_category    int32
traffic_source      int32
has_call_to_action  int64
post_datetime       datetime64[ns]
post_date           datetime64[ns]
post_hour           int64
day_of_week         int32
likes              int64
comments            int64
shares             int64
saves              int64
reach              int64
impressions         int64
engagement_rate     float64
followers_gained    int64
caption_length      int64
hashtags_count      int64
year               int64
month              int64
day                int64
hour               int64
dtype: object
```

```
X = X.drop(columns=['post_datetime', 'post_date'], errors='ignore')
```

16. BUILDING THE MODEL AND SPLITTING TRAIN AND TEST

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (23999, 23)
Testing set: (6000, 23)
```

17. BUILDING THE MODEL AND EVALUATING ACCURACY

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

Untitled - Jupyter Notebook

```
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
rf_model.fit(X_train, y_train)
print("Random Forest model trained successfully.")
```

Random Forest model trained successfully.

```
y_pred = rf_model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}")
```

Model Accuracy: 0.9028

CODE - SEMMA :

1. TO LOAD THE DATASET AND PRINT HEAD

```
#SAMPLE
import pandas as pd
df = pd.read_csv("Instagram_Analytics.csv")
```

```
df.head()
```

	post_id	account_id	account_type	follower_count	media_type	content_category	tra
0	IG0000001	7	brand	3551	reel	Technology	
1	IG0000002	20	creator	31095	image	Fitness	
2	IG0000003	15	brand	8167	reel	Beauty	
3	IG0000004	11	creator	9044	carousel	Music	
4	IG0000005	8	creator	15986	reel	Technology	

2. TO PRINT TAIL AND EXTRACT SAMPLE

```
df.tail()
```

	post_id	account_id	account_type	follower_count	media_type	co
29994	IG0029995	5	brand	10739	carousel	
29995	IG0029996	3	brand	10018	image	
29996	IG0029997	18	creator	7486	image	
29997	IG0029998	6	creator	10034	carousel	
29998	IG0029999	9	creator	6916	image	

5 rows × 23 columns

```
print("Shape:", df.shape)
```

Shape: (29999, 23)

```
sample_df = df.sample(frac=0.7, random_state=42)
```

3. TO PRINT INFO OF SAMPLE

```
#EXPLORE
sample_df.info()
```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 20999 entries, 2308 to 5419
Data columns (total 23 columns):
Column Non-Null Count Dtype
--- -
0 post_id 20999 non-null object
1 account_id 20999 non-null int64
2 account_type 20999 non-null object
3 follower_count 20999 non-null int64
4 media_type 20999 non-null object
5 content_category 20999 non-null object
6 traffic_source 20999 non-null object
7 has_call_to_action 20999 non-null int64
8 post_datetime 20999 non-null object
9 post_date 20999 non-null object
10 post_hour 20999 non-null int64
11 day_of_week 20999 non-null object
12 likes 20999 non-null int64
13 comments 20999 non-null int64
14 shares 20999 non-null int64
15 saves 20999 non-null int64
16 reach 20999 non-null int64
17 impressions 20999 non-null int64
18 engagement_rate 20999 non-null float64
19 followers_gained 20999 non-null int64
20 caption_length 20999 non-null int64
21 hashtags_count 20999 non-null int64
22 performance_bucket_label 20999 non-null object
dtypes: float64(1), int64(13), object(9)
memory usage: 3.8+ MB

4. TO PRINT DESCRIBE

```
sample_df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
account_id	20999.0	10.564074	5.752373	1.0	6.000	11.0000	16.0000	20.000
follower_count	20999.0	10272.369637	6685.471610	3083.0	5824.000	9044.0000	13798.0000	31095.000
has_call_to_action	20999.0	0.348636	0.476550	0.0	0.000	0.0000	1.0000	1.000
post_hour	20999.0	11.492357	6.897107	0.0	6.000	12.0000	17.0000	23.000
likes	20999.0	291.554455	333.088779	0.0	104.000	198.0000	366.0000	10632.000
comments	20999.0	8.628887	10.619238	0.0	3.000	6.0000	11.0000	339.000
shares	20999.0	14.636507	17.225875	0.0	5.000	10.0000	19.0000	516.000
saves	20999.0	43.015763	49.918302	0.0	15.000	29.0000	55.0000	1542.000
reach	20999.0	6264.382256	4988.506551	349.0	3048.500	4897.0000	7835.0000	73339.000
impressions	20999.0	8454.334635	6825.899216	400.0	4085.000	6569.0000	10608.5000	105203.000
engagement_rate	20999.0	0.042692	0.025745	0.0	0.024	0.0408	0.0573	0.271
followers_gained	20999.0	502.717558	290.300097	0.0	251.000	502.0000	755.0000	1000.000
caption_length	20999.0	120.065765	11.032281	79.0	113.000	120.0000	127.0000	166.000
hashtags_count	20999.0	7.982380	2.822675	0.0	6.000	8.0000	10.0000	21.000

5. TO ANALYSE NULL VALUES AND DUPLICATED ROWS

```
print(sample_df.isnull().sum())
```

```
post_id          0
account_id       0
account_type     0
follower_count   0
media_type       0
content_category 0
traffic_source   0
has_call_to_action 0
post_datetime    0
post_date        0
post_hour        0
day_of_week      0
likes            0
comments         0
shares           0
saves            0
reach           0
impressions      0
engagement_rate  0
followers_gained 0
caption_length   0
hashtags_count   0
performance_bucket_label 0
dtype: int64
```

```
print("Duplicate Rows:", sample_df.duplicated().sum())
```

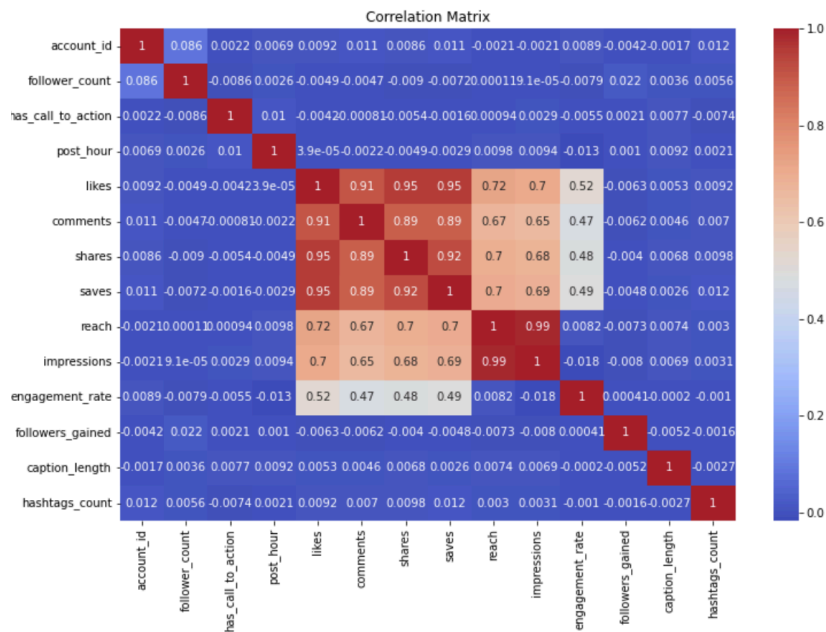
Duplicate Rows: 0

6. TO VISUALISE CORRELATION MATRIX

```
import matplotlib.pyplot as plt
import seaborn as sns

numeric_df = sample_df.select_dtypes(include=['number'])

plt.figure(figsize=(12,8))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

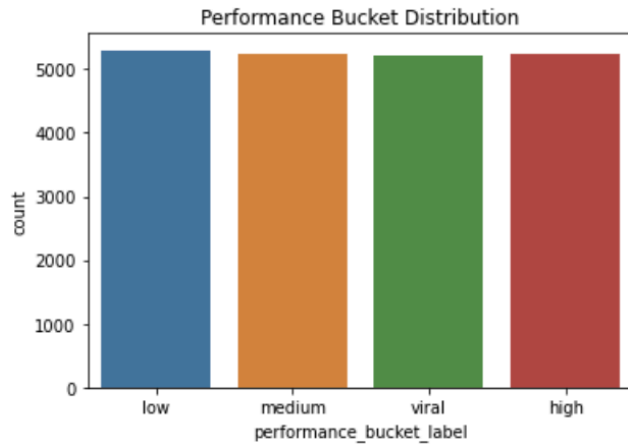


7. Performance Bucket Analysis

```
plt.figure(figsize=(6,4))

sns.countplot(
    data=sample_df,
    x="performance_bucket_label"
)

plt.title("Performance Bucket Distribution")
plt.show()
```

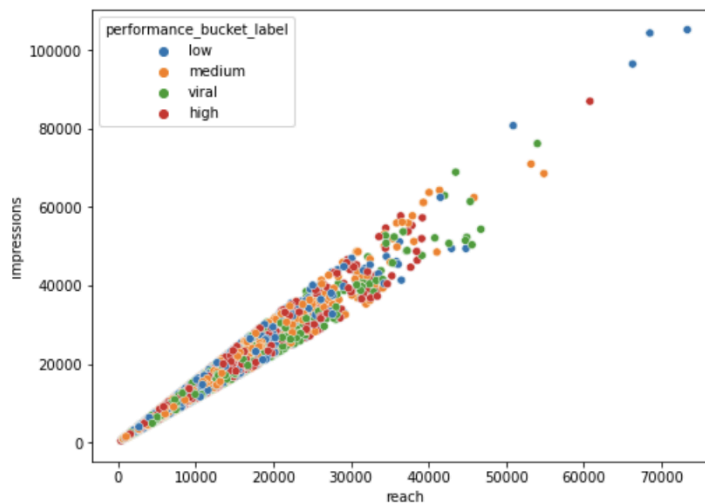


8. TO VISUALISE SCATTERPLOT

```
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=sample_df,
    x='reach',
    y='impressions',
    hue='performance_bucket_label'
)

plt.show()
```



9. MODIFYING AND CLEANING THE COLUMNS

```
#MODIFY
df_clean = sample_df.copy()

num_cols = df_clean.select_dtypes(include=['number']).columns

df_clean[num_cols] = df_clean[num_cols].fillna(
    df_clean[num_cols].median()
)

cat_cols = df_clean.select_dtypes(include=['object']).columns

df_clean[cat_cols] = df_clean[cat_cols].fillna(
    df_clean[cat_cols].mode().iloc[0]
)
```

```
df_clean.drop_duplicates(inplace=True)
```

```
df_clean['post_datetime'] = pd.to_datetime(df_clean['post_datetime'])
df_clean['post_date'] = pd.to_datetime(df_clean['post_date'])
```

```
df_clean["Year"] = df_clean["post_datetime"].dt.year
df_clean["Month"] = df_clean["post_datetime"].dt.month
df_clean["Day"] = df_clean["post_datetime"].dt.day
df_clean["Hour"] = df_clean["post_datetime"].dt.hour
df_clean["Weekday"] = df_clean["post_datetime"].dt.day_name()
```

```
df_clean.drop(
    columns=["post_id"],
    inplace=True
)
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_columns = df_clean.select_dtypes(include='object').columns

for col in categorical_columns:
    df_clean[col] = le.fit_transform(df_clean[col])
```

```
X = df_clean.drop("performance_bucket_label", axis=1)
y = df_clean["performance_bucket_label"]

X = X.drop(
    columns=["post_datetime", "post_date"],
    errors="ignore"
)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

10. BUILDING THE MODEL

```
#MODEL
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train,y_train)

print("Model Trained Successfully")
```

Model Trained Successfully

```
y_pred = rf.predict(X_test)
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Accuracy:", accuracy)
```

Accuracy: 0.9002380952380953

Result :

The program to perform basic mathematical operations is completed successfully

